An New Implementation Technique For The Dependently-Typed Lambda Lambda Calculus

Alexander Vandenbroucke
Unaffiliated
London, United Kingdom
alexander.vandenbroucke@gmail.com

ABSTRACT

Can you implement a dependently-typed lambda calculus using Turner's logical combinator technique? Yes. The reason this is interesting is that type checking a dependently-typed language inherently mixes type-checking with execution, because your types can be terms (and terms can be types), and hence have operational meaning. The solution presented here elaborates terms (or types, whichever) into combinator expressions on-the-fly during type checking. Only one extra combinator, T, is required in addition to Turner's 9 original combinators, S, K, I, P, B, C, S', B' and C' to implement a simple bi-directional type system (only the first four combinators are essential).

CCS CONCEPTS

• **Software and its engineering** → **Compilers**; **Semantics**.

KEYWORDS

combinators, dependent types, lambda calculus, type checking, type inference, bracket abstraction

ACM Reference Format:
Alexander Vandenbroucke. 2026. An New Implementation Technique For The Dependently-Typed Lambda Lambda Calculus. In *Proceedings of Symposium on Implementation and Application of Functional Languages (IFL2026).* ACM, New York, NY, USA, 7 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

Can you implement a dependently-typed lambda calculus using Turner's logical combinator technique? Yes. Here's an example of a polymorphic identity function:

$$(\lambda\alpha.\lambda x.x) : (\alpha : *) \to (x : \alpha) \to \alpha$$

And here is the combinator it is implemented by: `I`. This combinator implements the identity function directly. More interestingly, the type of the function, $(\alpha : *) \to (x : \alpha) \to \alpha$ elaborates to `P T (S P K)`. The `P` combinator constructs a function type, in other words, a **Pi** type. Rather than the usual infix-arrow syntax $(x : *) \to \cdots$, `P` is a prefix operator. We can thus read `P T (S P K)` as the type of a function expecting an argument α of type `T` (the type of types) and returning a value of type `S P K` α (note the dependency on the argument α). This reduces as follows: `S P K` $\alpha \Rightarrow$ `P` α (`K` α). This is another function type, this time expecting an argument x of type α, and producing a value of type `K` αx. This finally reduces to α: `K` $\alpha\ x \Rightarrow \alpha$, as expected of the identity function.

It should be obvious from the previous paragraph that a dependently-typed calculus has just as much interesting execution going on in its types as in its terms, if not more. This is because there is no (syntactic) distinction between terms and types: you can have types in your terms and terms in your types. The implication is that type checking and execution are inherently interwoven. Therefore, while in the middle of type checking, we need to generate an executable representation of the types against which terms are checked.[1] We choose to use Turner's logical combinators [4] because they are straightforward to implement, yet admit an easy path to decent performance via graph reduction. Most importantly, they make substitution trivial (which is usually tricky to get right) because the key idea of the combinators technique is to eliminate variables entirely.

Variable elimination is accomplished by the well-known bracket abstraction algorithm [3, 4]. The bracket abstraction algorithm for a dependently-typed lambda term that is presented here is only a modest generalisation from the original untyped version—though it is novel to the best of our knowledge.

The calculus implemented in this article is based on the language presented in "A tutorial implementation of a dependently typed lambda-calculus" [1] by Löh et al. They set out to prove that dependent types (and their implementation) need not be scary or hard to understand. It contains both a formal specification of a simple dependently typed lambda-calculus and a Haskell implementation of its abstract syntax, evaluation and type checking. The tutorial is excellent, and I would warmly recommend it to anyone interested in a core implementation of dependent types.

The remainder of this article is laid out as follows:

- We recall the syntax for both the simple dependently typed calculus introduced by the previously mentioned tutorial and then introduce the set of combinators needed for the implementation in this article: `S`, `K`, `I`, `P`, `T`, `B`, `C`, `S'`, `B'` and `C'` (Section 1). We also recall the reduction rules for these combinators (Section 2).
- Section 3 presents the bracket abstraction algorithm, extended with a rule to abstract function types, which is novel to the best of my knowledge.
- We introduce a bi-directional type system bearing a strong resemblance to the system introduced by the original tutorial.

[1] We generate combinators both during type checking and inference, but only checking is mentioned here for simplicity.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
IFL2026, 28 – 30 October, 2026, Gothenburg, Sweden
© 2026 Association for Computing Machinery.
ACM ISBN 978-x-xxxx-xxxx-x/YY/MM…$15.00
https://doi.org/10.1145/nnnnnnn.nnnnnnn

The key innovation is that our typing relations simultaneously perform an elaboration of the term that is being typed into a combinator expression. This means that whenever we needed to reduce a term in the original typing relation (necessitating re-quoting the term), we can now reduce the elaborated combinator expression instead. (Section 4)
- Section 5 gives some examples of typing and executing the new calculus.
- Section 6 concludes and discusses future work.

This article is written in literate Haskell and is executable. In addition to the functionality presented in the main text, the appendices contain code implementing parsing, pretty printing, a simple REPL and QuasiQuoters [2].

1 SYNTAX

To begin, let us define the (abstract) syntax of the calculus and the combinators. Since this is a dependently typed calculus, there is no syntactic distinction between types and terms: both are captured by the `Term` GADT. However, there is a separation between terms whose type can be inferred, and whose type can only be checked. For this reason, `Term` is indexed by a `Mode`. In all other respects the syntax should be entirely unsurprising. The constructors are annotated with their surface level syntax (e.g., `*` for `Typ`, `\x.t` for `Lam x t`, etc.).

```
data Mode = Infer | Check

data Term (mode :: Mode) where
  Var :: String -> Term Infer
  -- ^ Variable, x
  Ann :: Term Check -> Term Check -> Term Infer
  -- ^ Type annotation, t : t
  App :: Term Infer -> Term Check -> Term Infer
  -- ^ Application, t t
  Typ :: Term Infer
  -- ^ Type of Types, *
  Fun :: String -> Term Check -> Term Check -> Term Infer
  -- ^ Function Type, (x:t) -> t

  Lam :: String -> Term Check -> Term Check
  -- ^ Lambda, \x.t
  Inf :: Term Infer -> Term Check
  -- ^ Embed Inferrable Term, t
```

The combinator syntax contains standard `S`, `K`, `I`, `B` and `C` combinators; as well as `P` for (dependent) function types (Π-types); `T` for the type of types; `V` for variables, and lastly an infix application constructor `:@:`. The `S'`, `B'` and `C'` combinators and the "constant" application constructors `:<@:`, `:@>:` and `:<@>:` are used for the more efficient bracket abstraction algorithm, described by Turner [3]. They can be ignored on a first reading.

```
data Combinator where
  S :: Combinator
  K :: Combinator
  I :: Combinator
  B :: Combinator
  C :: Combinator
  P :: Combinator            -- ^ Pi-type
  T :: Combinator            -- ^ Type
  V :: String -> Combinator  -- ^ Variable
  (:@:) :: Combinator -> Combinator -> Combinator
  -- ^ Application

  S' :: Combinator
  B' :: Combinator
  C' :: Combinator
  (:<@: ) :: Combinator -> Combinator -> Combinator
  ( :@>:) :: Combinator -> Combinator -> Combinator
  (:<@>:) :: Combinator -> Combinator -> Combinator
  deriving Eq

infixl :@:
```

For reduction, these extra application constructors are irrelevant. It is thus useful to "normalise" them away at the top-level of a combinator expression:

```
wnf :: Combinator -> Combinator
wnf (a :<@:  b) = a :@: b
wnf (a  :@>: b) = a :@: b
wnf (a :<@>: b) = a :@: b
wnf a = a

pattern a :*: b <- (wnf -> a :@: b)
```

Here `:*:` is a pattern synonym for "any application constructor", think of the `*` as a wildcard for any combination of `<`, `>` and `@`.

The extra application constructors are also irrelevant when comparing two combinator expressions, so we define an equality operator which ignores this difference:

```
(=*=) :: Combinator -> Combinator -> Bool
a =*= b | a == b = True
(a :*: b) =*= (c :*: d) = a =*= c && b =*= d
_ =*= _ = False
infix 4 =*=
```

2 COMBINATOR REDUCTION

Combinators have the usual reduction rules. The `reduce` function (written in formulae as ⇒) performs a single step of reduction if possible. The first 5 clauses implement standard reduction rules for `S`, `I`, `K`, `B`, `C`:

```
reduce :: Combinator -> Maybe Combinator
reduce = \case
  S :*: f :*: g :*: x -> Just (f :@: x :@: (g :@: x))
  I :*: x             -> Just x
  K :*: x :*: y       -> Just x
  B :*: f :*: g :*: x -> Just (f :@: (g :@: x))
  C :*: f :*: g :*: x -> Just (f :@: x :@: g)
```

The next three clauses implement the following reduction rules for `S'`, `B'` and `C'`:

$$S'\ k\ f\ g\ x \Rightarrow k\ (f\ x)\ (g\ x)$$
$$B'\ k\ f\ g\ x \Rightarrow k\ f\ (g\ x)$$
$$C'\ k\ f\ g\ x \Rightarrow k\ (f\ x)\ g$$

Again, feel free to ignore these on a first reading.

```
  S' :*: k :*: f :*: g :*: x->Just (k :@: (f :@: x) :@: (g :@: x))
  B' :*: k :*: f :*: g :*: x->Just (k :@: f :@: (g :@: x))
  C' :*: k :*: f :*: g :*: x->Just (k :@: (f :@: x) :@: g)
```

Finally, if none of the previous cases match, we recurse down the left argument until we find the left-most expression that can

be reduced. If reduction is not possible there, we try the right argument.

```
  f :*: g | Just f' <- reduce f -> Just (f' :@: g)
          | Just g' <- reduce g -> Just (f  :@: g')
  _                             -> Nothing
```

This implies, for instance, that in `P x y`, `x` and `y` can be reduced.

The extension of $\Rightarrow$ to multiple steps, denoted $\overset{*}{\Rightarrow}$, is implemented by `reduce'`:

```
reduce' :: Combinator -> Combinator
reduce' c = maybe c reduce' (reduce c)
```

3 BRACKET ABSTRACTION

The bracket abstraction algorithm is also standard, so we will not repeat most of the rules here, except that dependent function types $(y : E_1) \to E_2$ are abstracted by the following rule (using $[x]E$ to denote abstracting E over x):

$$\begin{aligned}[x]((y : E_1) \to E_2) &= [x](P\ E_1\ [y]E_2) \\ &= S\ [x](P\ E_1)\ [x]([y]E_2) \\ &= S\ (S\ (K\ P)\ [x]E_1)\ [x]([y]E_2)\end{aligned}$$

That is, the body of the function type is first abstracted over y, and then the `P` combinator is applied to both E_1 and $[x]E_2$. This presents a slight problem: since E_1 is not a combinator expression, `P` cannot be applied to it directly, so $P\ E_1$ is nonsense. This will be addressed shortly. The entire expression is then abstracted over x.

Now, recall the bracket-abstraction rule for application:

$$[x](E_1\ E_2) = S\ [x]E_1\ [x]E_2$$

Applying this rule twice mends the notation.

For convenience, abstraction is defined in a type-class. This allows ad-hoc polymorphism in the `abstract` function, because we need to be able to abstract both terms and combinators expressions.

```
class Abstraction e where
  abstract :: String -> e -> Combinator

instance Abstraction (Term m) where
  abstract x Typ           = K @@ T
  abstract x (App e1 e2)   = S @@ abstract x e1
                               @@ abstract x e2
  abstract x (Lam y e)     = abstract x (abstract y e)
  abstract x (Ann e _)     = abstract x e
  abstract x (Inf e)       = abstract x e
  abstract x (Fun y e1 e2) = S @@ (S @@ (K @@ P)
                                     @@ abstract x e1)
                                  @@ abstract x (abstract y e2)
  abstract x (Var x') | x == x' = I
                      | otherwise = K @@ V x'

instance Abstraction Combinator where
  abstract x (V x') | x == x' = I
  abstract x (e1 :*: e2)      = S @@ abstract x e1
                                  @@ abstract x e2
  abstract x c                = K @@ c
```

The `@@` operator is a smart constructor for application. It implements a more efficient abstraction. Like `:@:` it binds most tightly to the left. On a first reading, the reader may pretend that `@@` is simply `:@:`, and skip the remainder of this section.

The first four cases of `@@` replace degenerate applications of `S` with applications of `K`, `B` and `C`. Degenerate means that the variable being abstracted does not occur in one or both of the sub-expressions.

The next four cases replace applications of `S`, `B` and `C` with the special combinators `S'`, `B'` and `C'`. This eliminates the blow-up that might occur after repeated abstraction with a series of variables—this happens if you abstract a sequence of nested lambdas.

The final four clauses define the special application operator `@@` tracking which operands are composed entirely of constants. The `<` or `>` points to the arguments that are constant.

```
(@@) :: Combinator -> Combinator -> Combinator
(S :*: (K :*: a)) @@ (K :*: b) = K @@ (a @@ b)
(S :*: (K :*: a)) @@ I         = a
(S :*: (K :*: a)) @@ b         = B @@ a @@ b
(S :*: a)         @@ (K :*: b) = C @@ a @@ b

(S :*: ((B :<@>: k) :*: a)) @@ b = S' @@ k @@ a @@ b
(B :*: (k :<@: a))          @@ b = B' @@ k @@ a @@ b
(B :*: (k :<@>: a))         @@ b = B' @@ k @@ a @@ b
(C :*: ((B :<@>: k) :*: a)) @@ b = C' @@ k @@ a @@ b

a @@ b | constant a, constant b = a :<@>: b
       | constant a             = a :<@:  b
       | constant b             = a  :@>: b
       | otherwise              = a  :@:  b
       where
         constant (V _)       = False
         constant (_  :@>: _) = False
         constant (_ :<@:  _) = False
         constant (_  :@:  _) = False
         constant _ = True

infixl @@
```

4 TYPING

We now turn to implementing a bi-directional dependently typed calculus. Declaratively, the rules are not essentially different from the Lambda-Pi calculus, except that we also elaborate the terms into combinator expressions while type-checking. Thus, type inference results in two combinator expressions: the type (`Typ`) and an elaboration (`Elaboration`), essentially capturing the static and dynamic semantics of a term. The type must be irreducible, and follows from the Lambda-Pi bi-directional type system. The elaboration follows bracket abstraction (more or less) and may admit additional reduction.

```
type Typ         = Combinator -- irreducible
type Elaboration = Combinator
data Inferred    = Typ :~> Elaboration deriving Show
```

As a preliminary, we define simple environments:

```
type Env = [(String,Typ)]
```

Given variable x and type t, extending an environment Γ results in a new environment $x : t_x, \Gamma$ where t_x is the result of substituting all variables currently in E into t. Therefore t_x should not have any free variables.

```
extend :: String -> Combinator -> Env -> Env
extend x c env = (x,t') : env
  where
    t' = reduce' (foldr (\(y,c) -> (@@ c) . abstract y) c env)
```

4.1 Type Inference

In the following, $\Gamma \vdash e \Rightarrow t \rightsquigarrow c$ will denote that a type t is inferred for term e under environment Γ and elaborates to c and $\Gamma \vdash e \Leftarrow t \rightsquigarrow c$ means that e has type t under environment G and elaborates to c. The arrows $\Rightarrow$ and $\Leftarrow$ are meant to suggest that information flows from the term to the type and vice versa. As before, abstraction of e with respect to variable x is written as $[x]e$, and $e \overset{*}{\Rightarrow} e'$ means that the expression e reduces to e', where e' is irreducible. Each typing rule is given together with the corresponding clause of `infer` or `check` implementing that rule.

Variable. A variable's type is inferred by looking it up in the environment.

$$\frac{x : t \in \Gamma}{\Gamma \vdash x \Rightarrow t \rightsquigarrow x}$$

```
infer :: Env -> Term Infer -> Either String Inferred
infer env (Var x) = case lookup x env of
  Nothing -> Left  $ "unbound variable: " <> x
  Just t  -> Right $ t :~> V x
```

Annotation. For a type annotation, we must first check that the expression on the right-hand side of the annotation is indeed a type, i.e., has type $*$. As a by-product of type checking, we also get the elaboration. This is a combinator expression, which we must reduce further to obtain a type. Finally, we check that the expression on the right-hand side of the annotation indeed has this type.

$$\frac{\Gamma \vdash r \Leftarrow T \rightsquigarrow r' \qquad r' \overset{*}{\Rightarrow} t \qquad \Gamma \vdash e \Leftarrow t \rightsquigarrow c}{\Gamma \vdash e : r \Rightarrow t \rightsquigarrow c}$$

```
infer env (Ann e r) = do
  r' <- check env r T
  let t = reduce' r'
  c <- check env e t
  Right $ t :~> c
```

Type. Inferring the type of $*$ is entirely trivial—though perhaps a bit controversial: it is `T`.

$$\frac{}{\Gamma \vdash * \Rightarrow T \rightsquigarrow T}$$

```
infer _ Typ = Right $ T :~> T
```

Application. Inferring the type of an application starts by inferring the type of the left- hand expression. This must be a function type (`P`-combinator). Next, we check that the argument (the right-hand expression) has the expected argument type. As a by-product, we now have the elaboration of the argument available, which we apply to the function type's body to calculate the return type.

$$\frac{\Gamma \vdash e_1 \Rightarrow P\ t_2\ f \rightsquigarrow c_1 \qquad \Gamma \vdash e_2 \Leftarrow t_2 \rightsquigarrow c_2 \qquad f\ c_2 \overset{*}{\Rightarrow} t}{\Gamma \vdash e1\ e2 \Rightarrow t \rightsquigarrow c_1\ c_2}$$

```
infer env (App e1 e2) = do
  t1 :~> c1 <- infer env e1
  case t1 of
    P :*: t2 :*: f -> do
      c2 <- check env e2 t2
      let t = reduce' (f @@ c2)
      Right $ t :~> (c1 @@ c2)
    _ -> Left $
      "Could not match expected type " <>
      show (Fun "x" (Inf (Var "A")) (Inf (Var "B"))) <>
      " with actual type " <>
      show t1
```

Function Type. Lastly, to infer the type of a function type, we check that the argument type is in fact a type. Then we check that the return type is a type, under an environment extended with the argument variable and the elaboration of the argument type. The elaboration of the function is thus the P combinator applied to the argument type c_1 and the return type c_2 *abstracted over the argument variable*. The elaboration is allowed to be reducible, so there is no need to reduce c_1 and c_2.

$$\frac{\Gamma \vdash t_1 \Leftarrow * \rightsquigarrow c_1 \qquad (\mathit{extend}\ x\ c_1\ \Gamma) \vdash t_2 \Leftarrow * \rightsquigarrow c_2}{\Gamma \vdash (x : t_1) \to t_2 \Rightarrow * \rightsquigarrow P\ c_1\ [x]c_2}$$

```
infer env (Fun x t1 t2) = do
  c1 <- check env t1 T
  c2 <- check (extend x c1 env) t2 T
  Right $ T :~> (P @@ c1 @@ abstract x c2)
```

4.2 Type Checking

We can check all inferable terms simply by inferring their type, and then verifying that the inferred and the expected combinator expressions are the same.

$$\frac{\Gamma \vdash e \Rightarrow t' \rightsquigarrow c \qquad t \overset{*}{=} t'}{\Gamma \vdash e \Leftarrow t \rightsquigarrow c}$$

```
check :: Env -> Term Check -> Typ -> Either String Elaboration
check env (Inf e) t = do
  t' :~> e' <- infer env e
  if t =*= t' then
    Right e'
  else
    Left $
      "Type mismatch: Could not match expected type "
      <> show t <> " with inferred type " <> show t'
```

Lambda. A lambda term can only be checked successfully against a function type. The procedure is to check the body against the calculated return type, in an environment extended with the bound variable. This return type is calculated by applying the body of the function type to the same type with which we extended the environment. In this type all variables that are inside the environment will have been instantiated.

$$\frac{x : t_x, \Gamma = \mathit{extend}\ x\ t_1\ \Gamma \qquad f\ t_x \overset{*}{\Rightarrow} t_2 \qquad x : t_x, \Gamma \vdash e \Leftarrow t_2 \rightsquigarrow c}{\Gamma \vdash \lambda x.e \Leftarrow P\ t_1\ f \rightsquigarrow [x]c}$$

```
check env (Lam x e) t = do
  case t of
    P :*: t1 :*: f -> do
      let (_,tx):_ = extend x t1 env
          t2 = reduce' (f @@ tx)
      abstract x <$> check ((x,tx):env) e t2
      -- should we substitute into the body here?
      -- does lambda-pi have the same flaw?
    _ ->
      Left $
        "Could not match type " <> show t <>
        " while checking a lambda abstraction."
```

5 EXAMPLES

The remaining code in this article implements pretty printing, parsing and a small REPL, that, while useful, are not particularly interesting. However, the REPL will serve as a vehicle for some small examples.

Typing an (inferable) term at the REPL causes it to parse, type and then reduce that term, for example:

```
> *
T -> T
 : T
```

This means that the string `"*"` is elaborated into the term T. The `=> T` means that this elaboration reduces to T, and `: T` means that it has type T. Let us look at an example that uses more than one combinator next:

```
> (\x.x) : (x:*) -> *
I => I
 : P T (K T)
```

The term $\lambda x.x : (x : *) \longrightarrow *$ elaborates to I, which is irreducible and it has type $P\ T\ (K\ T)$, that is, a function (P) accepting some argument x of type T (i.e., x is a type) and returning a value of type $K\ T\ x \Rightarrow T$ (also a type). Contrast with the following:

```
> (\f.f *) : (f:(x:*) -> *) -> *
C I T => C I T
 : P (P T (K T)) (K T)
```

Here the expression takes a function $f : (x : *) \longrightarrow *$ as argument, and applies it to $*$. The type $P\ (P\ T\ (K\ T))\ (K\ T)$ is a function, taking a function f (see above) as an argument and returning $K\ T\ f \Rightarrow T$.

So far, all combinator expressions have been irreducible. Let's change that by applying (`\f.f *`) to the identity function:

```
> ((\f.f *) : (f:(x:*) -> *) -> *) (\x.x)
C I T I => T
 : T
```

The expression now elaborates to $C\ I\ T\ I \Rightarrow I\ I\ T \Rightarrow I\ T \Rightarrow T$. The overall expression has type T as expected. Finally, we are not restricted to just types ($*$) as arguments. Here is an identity function that is polymorphic in the type A:

```
> (\A.\x.x) : (A:*) -> (x:A) -> A
K I => K I
 : P T (S P K)
```

Applying the return type to some type A, we get $S\ P\ K\ A \Rightarrow P\ A\ (K\ A)$, that is, a function where we expect an argument, say x, of type A, and then the return type is $K\ A\ x \Rightarrow A$.

We can also introduce a higher-order type T, which we can then apply to a matching type:

```
> (A : (B : *) -> *) -> A *
P (P T (K T)) (C I T) => P (P T (K T)) (C I T)
 : T
```

The produced expression certainly looks impressive. We cannot just pass any old type to A:

```
> (C : (B : *) -> *) -> (A : (B : *) -> *) -> A C
Type mismatch: Could not match expected type T with
inferred type P T (K T)
```

Have we really unified terms and types? Let's define a function with a non-trivial type:

```
> (\x.x) : (((\t.(x:t) -> t) : (t : *) -> *) *)
I => I
 : P T (K T)
```

We have a function in a type, so yes!

6 CONCLUSION

Turner's logical combinator technique is a flexible method for the implementation of functional programming languages. As a testament to this, we have shown that only a modest extension is required to adapt it to a dependently-typed setting. This setting is particularly interesting, because type checking in this setting is inherently mixed with evaluation. This has guided us towards an extension of the existing bi-directional type checking algorithm that elaborates terms or types into combinators on-the-fly.

Future Work. Future work consists of two lines of research: First, on the theoretical side, formalising the meta-theory, including a proof that the type-directed elaboration is correct with respect to the usual operational semantics is an important direction for future work. Given the moderate size and unsurprising nature of the type system, we expect standard proof techniques to be sufficient.

Secondly, practical combinator implementations rely on graph reductions to obtain good performance. It is an open question how we can wed a graph reduction engine, an essentially imperative piece of machinery, to the purely functional implementation of type checking and inference.

REFERENCES

[1] Andres Löh, Conor McBride, and Wouter Swierstra. A tutorial implementation of a dependently typed lambda calculus. *Fundam. Informaticae*, 102(2):177–207, 2010.

[2] Geoffrey Mainland. Why it's nice to be quoted: quasiquoting for haskell. In *Proceedings of the ACM SIGPLAN Workshop on Haskell Workshop*, Haskell '07, page 73–82, New York, NY, USA, 2007. Association for Computing Machinery.

[3] D. A. Turner. Another algorithm for bracket abstraction. *J. Symb. Log.*, 44(2):267–270, 1979.

[4] D. A. Turner. A new implementation technique for applicative languages. *Softw. Pract. Exp.*, 9(1):31–49, 1979.

7 APPENDIX: PRETTY PRINTING

The built-in Show class is good enough to insert the right parentheses when appropriate:

```
instance Show (Term m) where
  showsPrec d = \case
    Var x -> str x
    Typ   -> str "*"
    Inf e -> showsPrec d e
    Ann e t -> showParen (d > ann_d) $
      showsPrec (ann_d + 1) e . str " : " . showsPrec fun_d t
    App e1 e2 -> showParen (d > app_d) $
```

```
      showsPrec (app_d + 1) e1
      . str " "
      . showsPrec (app_d + 1) e2
    Lam x e -> showParen (d > lam_d) $
      str "\\" . str x . str "." . showsPrec lam_d e
    Fun x t1 t2 -> showParen (d > fun_d) $
      str "(" . str x . str  ":" . showsPrec (fun_d + 1) t1
      . str ") -> "
      . showsPrec fun_d t2
    where
      str x r = x ++ r
      lam_d = 1 :: Int
      fun_d = 2 :: Int
      ann_d = 3 :: Int
      app_d = 4 :: Int

instance Show Combinator where
  showsPrec d = \case
    S -> str "S"
    K -> str "K"
    I -> str "I"
    B -> str "B"
    C -> str "C"
    P -> str "P"
    T -> str "T"
    V x -> (x ++)
    c1  :@:  c2 -> app c1 c2
    c1 :<@:  c2 -> app c1 c2
    c1  :@>: c2 -> app c1 c2
    c1 :<@>: c2 -> app c1 c2
    S' -> str "S'"
    B' -> str "B'"
    C' -> str "C'"
    where
      app c1 c2 = showParen (d > 1) $
        showsPrec 1 c1 . (" " ++) . showsPrec 2 c2
      str x r = x ++ r
```

8 APPENDIX: PARSING

The parser recognises the following grammar:

```
Expr := \V -> Abs | App
Type := (x:Expr) -> Typ | Ann
Ann  := App : Type | App
App  := App Atom | Atom
Atom := var | * | (Expr)
```

The approach followed here was inspired by ”Design Patterns for Parser Combinators (Functional Pearl)”.

```
data Expr  = ELam String Expr       | OfType EType deriving Show
data EType = EFun String Expr EType | OfAnn  EAnn  deriving Show
data EAnn  = EAnn EApp EType        | OfApp  EApp  deriving Show
data EApp  = EApp EApp EAtom        | OfAtom EAtom deriving Show
data EAtom = EVar String | ETyp | EPar Expr        deriving Show

expr :: ReadP Expr
expr = (<++ (OfType <$> etype)) $ do
  symbol "\\"
  x <- variable
  symbol "."
  e <- expr
  pure $ ELam x e

eann :: ReadP EAnn
eann = do
  e <- eapp
  ((EAnn e <$ symbol ":" <*> etype) <++ pure (OfApp e))

etype :: ReadP EType
etype = (<++ (OfAnn <$> eann)) $ do
  symbol "("
  x <- variable
  symbol ":"
  e1 <- expr
  symbol ")"
  symbol "->"
  e2 <- etype
  pure (EFun x e1 e2)

eapp :: ReadP EApp
eapp = infixl1 OfAtom eatom (pure EApp)

eatom :: ReadP EAtom
eatom =
      (ETyp <$  symbol "*")
  <++ (EVar <$> variable)
  <++ (EPar <$> (symbol "(" *> expr <* symbol ")"))


lexeme :: ReadP a -> ReadP a
lexeme p = p <* skipSpaces

symbol :: String -> ReadP String
symbol = lexeme . string

variable :: ReadP String
variable =
  lexeme $
    (:) <$> satisfy (liftA2 (||) isAlpha (=='_'))
        <*> munch isAlphaNum

infixl1 :: (a -> b) -> ReadP a -> ReadP (b -> a -> b) -> ReadP b
infixl1 wrap p op = (wrap <$> p) <**> rest
  where
    rest = (flip (.) <$> (flip <$> op <*> p) <*> rest) <++ pure id
    fx <**> ff = do
      x <- fx
      f <- ff
      pure (f x)

class ToTerm e where
  toTerm :: e -> Maybe (Term Check)

instance ToTerm Expr where
  toTerm (ELam x e) = Lam x <$> toTerm e
  toTerm (OfType e)  = toTerm e

instance ToTerm EAnn where
  toTerm (EAnn e t) = Inf <$> (Ann <$> toTerm e <*> toTerm t)
  toTerm (OfApp t)  = toTerm t

instance ToTerm EType where
  toTerm (EFun x t1 t2) = Inf <$> (Fun x <$> toTerm t1 <*> toTerm t2)
  toTerm (OfAnn e)      = toTerm e
```

```
instance ToTerm EApp where
  toTerm (EApp e a) = do
    Inf t <- toTerm e
    Inf . App t <$> toTerm a
  toTerm (OfAtom a) = toTerm a

instance ToTerm EAtom where
  toTerm (EVar x) = Just (Inf (Var x))
  toTerm ETyp     = Just (Inf Typ)
  toTerm (EPar e) = toTerm e

parse :: String -> Either String (Term Check)
parse = result . readP_to_S (skipSpaces *> expr <* eof) where
  result [(e,"")] | Just t <- toTerm e = Right t
  result _                             = Left "Syntax error!"
```

9 APPENDIX: SIMPLE REPL

```
main :: IO ()
main =
  let
    eval str = either id id $ do
      t <- parse str
      case t of
        Lam{} -> Left "The top-level term must be inferable!"
        Inf t -> do
          (t :~> c) <- infer [] t
          Right $
            show c <> " => " <> show (reduce' c)
            <> "\n : " <> show t
    go = do
      putStr "> "
      getLine >>= \str -> do
        case str of
          ":h" -> do
            putStrLn $ "Usage: <expression> | :l <file> | :h | :q"
            go
          ":q" -> pure ()
          (words -> [":l", file]) -> do
            try (readFile file) >>=
               putStrLn . either (displayException @IOError) eval
            go
          _ -> do
            putStrLn $ eval str
            go
  in go
```

10 APPENDIX: QUASI QUOTER

Scaffolding code

```
deriving instance Lift (Term Infer)
deriving instance Lift (Term Check)
deriving instance Lift Combinator

qq :: String -> (String -> Q Exp) -> QuasiQuoter
qq name exp =
  QuasiQuoter
  { quoteExp  = exp
  , quotePat  = invalid "pattern"
  , quoteType = invalid "type"
  , quoteDec  = invalid "declaration"
  }
  where
    invalid :: String -> String -> Q a
    invalid ctx _ =
      fail $
        name
        <> "quasi-quoter cannot be used in a "
        <> ctx <> " context!"
```

Quoting of expressions `lpc` or, inferable expression `lpci`:

```
lpc :: QuasiQuoter
lpc = qq "lpc" $ \str -> case parse str of
  Left  err  -> fail $ "lpc quasi-quoter parse error: " <> err
  Right term -> [|term|]

lpci :: QuasiQuoter
lpci = qq "lpci" $ \str -> case parse str of
  Left  err ->
    fail $ "lpci quasi-quoter parse error: " <> err
  Right Lam{} ->
    fail "lpci quasi-quoter: quoted expression is not inferable"
  Right (Inf term) -> [|term|]
```

A quoter that any allows expressions of type $*$ to be quoted:

```
typ :: QuasiQuoter
typ = qq "typ" $ \str -> case parse str of
  Left err -> fail $ "typ quasi-quoter parse error: " <> err
  Right t -> case check [] t T of
    Left err ->
      fail $
        "typ quasi-quoter: quoted expression is not a type: " <> err
    Right c ->
      let r = reduce' c in [|r|]
```